
Software Requirements Specification

Untuk

Sistem Pemesanan Berbasis QR MyGerai

Versi 1.0

Disusun Oleh

Satria Permata Sejati - 231524026

CV Garuda Infinity Kreasindo

6 September 2026

Daftar Isi

1. PENDAHULUAN	1
1.1 Tujuan.....	1
1.2 Konvensi Dokumen	1
1.3 Target dan Saran Pembacaan.....	1
1.4 Lingkup Produk	1
1.5 Referensi.....	2
2. DESKRIPSI UMUM.....	2
2.1 Perspektif Produk	2
2.2 Fungsi Produk.....	3
2.3 Karakteristik Pengguna.....	4
2.4 Lingkungan Operasional.....	5
2.5 Batasan Desain dan Implementasi	5
2.6 Asumsi & Dependensi	6
3. FITUR SISTEM.....	6
3.1 Fitur 1: Melakukan Pemesanan dan Pembayaran	6
3.2 Fitur 2: Memantau Status Pesanan	10
3.3 Fitur 3: Mendaftar sebagai Pedagang	12
3.4 Fitur 4: Mengelola Katalog Item	14
3.5 Fitur 5: Memproses Pesanan Masuk.....	16
3.6 Fitur 6: Mengelola QR Lapak.....	18
3.7 Fitur 7: Meninjau Pendaftaran Pedagang	19
3.8 Fitur 8: Mengatur Konfigurasi Platform.....	21
3.9 Fitur 9: Memantau Transaksi dan Saldo Pedagang	23
3.10 Fitur 10: Mencatat Pencairan Saldo Pedagang	24
4. NON-FUNCTIONAL REQUIREMENTS.....	26
4.1 Performance Requirements.....	26
4.2 Safety Requirements.....	26
4.3 Security Requirements.....	27
4.4 Software Quality Attributes.....	29
4.5 Business Rules	31
5. REQUIREMENTS LAINNYA	31
LAMPIRAN A: GLOSARIUM	32

1. Pendahuluan

1.1 Tujuan

Dokumen *Software Requirements Specification* (SRS) ini bertujuan mendefinisikan secara terperinci spesifikasi kebutuhan perangkat lunak untuk Sistem Pemesanan Berbasis QR MyGera Versi 1.0. Dokumen ini menjabarkan ruang lingkup fitur keseluruhan sistem secara utuh, mencakup subsistem antarmuka untuk Pembeli, Pedagang, dan Admin, beserta proses otomatis yang dijalankan sistem.

Sasaran rilis pertama (V1) adalah alur inti pemesanan — pindai QR Lapak, pilih Item, *checkout*, bayar melalui QRIS — dengan pembayaran yang untuk sementara *disimulasikan*. Integrasi *payment gateway* sungguhan direncanakan sebagai tahap lanjutan dan turut didokumentasikan agar arsitektur V1 sudah siap menampungnya.

1.2 Konvensi Dokumen

Dokumen ini disusun dengan tata bahasa Indonesia yang baku dan mengikuti kerangka standar IEEE untuk SRS sebagaimana diuraikan Karl E. Wiegers. Istilah teknis berbahasa Inggris dipertahankan dalam bentuk aslinya dan ditulis miring (*italic*), misalnya *marketplace*, *payment gateway*, *webhook*, *server*, dan *checkout*. Istilah domain khas MyGera (Aplikator, Admin, Pedagang, Lapak, Pembeli, Item, Keranjang, Pesanan, Kode Pesanan, Biaya Layanan, Saldo Pedagang, Pencairan) ditulis dengan huruf kapital di awal dan maknanya baku sesuai Lampiran A.

Penomoran kebutuhan fungsional memakai format FR-XX dan kebutuhan non-fungsional memakai format NFR-XX. Setiap pernyataan kebutuhan memiliki tingkat prioritas tersendiri (Tinggi, Menengah, atau Rendah) yang berdiri sendiri dan tidak diwariskan dari hierarki bagian di atasnya.

1.3 Target dan Saran Pembacaan

Dokumen ini ditujukan bagi pemangku kepentingan pada tahap perancangan, pengembangan, dan pengoperasian MyGera: pengembang perangkat lunak (*developer*), penguji (*tester*), pemilik produk/Aplikator, serta Admin operasional.

Seluruh pembaca disarankan memulai dari Bab 1 untuk memahami visi dan batasan produk, lalu Bab 2 untuk gambaran umum arsitektur dan klasifikasi pengguna. Tim teknis (*developer* dan *tester*) diwajibkan menelaah Bab 3, Bab 4, dan Bab 5 secara saksama karena bagian tersebut memuat daftar kebutuhan fungsional dan non-fungsional yang harus diimplementasikan dan diuji.

1.4 Lingkup Produk

Sistem Pemesanan Berbasis QR MyGera adalah aplikasi web yang memungkinkan pedagang kecil pinggir jalan dan pasar (penjual bakso, batagor, cakue, pakaian, dan sejenisnya) menerima

pesanan yang sudah dibayar secara *real-time*, tanpa antri dan tanpa salah catat. Alur intinya mengadaptasi ESB Order namun disederhanakan drastis untuk skala pedagang kaki lima: Pembeli memindai QR Lapak, memilih Item, mengisi Nama, lalu membayar melalui QRIS; Pesanan otomatis masuk ke *dashboard* Pedagang setelah pembayaran terkonfirmasi.

MyGera menerapkan model pungutan berupa Biaya Layanan tetap (*flat fee*) — default Rp1.000 per Pesanan sukses — yang dipotong dari bagian Pedagang, bukan ditambahkan ke tagihan Pembeli. Penyelesaian dana memakai Model Agregator: seluruh pembayaran masuk ke satu akun milik Aplikator, lalu diteruskan ke Pedagang melalui Pencairan. Pada V1 pembayaran masih *disimulasikan* (*MockPaymentProvider*), Pembeli tidak memerlukan akun dan cukup mengisi Nama, serta tidak perlu memasang aplikasi apa pun.

Visi produk: memberi pedagang informal alat pencatatan pesanan dan pembayaran nontunai yang sesederhana mungkin, ringan diakses dari ponsel kelas bawah pada jaringan lambat.

1.5 Referensi

- IEEE Computer Society. (1998). *IEEE Recommended Practice for Software Requirements Specifications* (IEEE Std 830-1998).
- Wiegers, K. E., & Beatty, J. (2013). *Software Requirements* (Edisi Ketiga). Microsoft Press.
- Larman, C. (2004). *Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design and Iterative Development* (Edisi Ketiga). Prentice Hall.
- ESB. *ESB Order*. Diakses dari <https://www.esb.id/id/solusi/produk/order>.
- Bank Indonesia. (2019). *Ketentuan Implementasi Standar Nasional Quick Response Code untuk Pembayaran* (PADG No. 21/18/PADG/2019).
- Dokumen ground truth internal proyek MyGera: PRD.md, ARSITEKTUR-SISTEM.md, DATA-MODEL.md, TEKNOLOGI.md, GLOSSARY.md, BEST-PRACTICES.md, dan RULES.md.

2. Deskripsi Umum

2.1 Perspektif Produk

MyGera adalah produk perangkat lunak baru dan mandiri (*self-contained*), bukan kelanjutan atau modul dari sistem yang sudah ada. Secara arsitektur, MyGera berupa satu aplikasi *monolith* Next.js (*App Router*) yang melayani tiga muka pengguna — Pembeli, Pedagang, dan Admin — sebagai *route group* berbeda di dalam aplikasi yang sama, berbagi satu basis data PostgreSQL. Aplikasi dan basis data sama-sama *self-hosted* di server milik Aplikator, melalui Dokploy di balik Cloudflare Tunnel.

Basis data hanya diakses melalui kode server tepercaya (*Server Action* dan *Route Handler*); peramban tidak pernah terhubung langsung ke basis data. Notifikasi Pesanan ke Pedagang dan pembaruan status ke Pembeli memakai *polling* berkala dari peramban, bukan koneksi *realtime* khusus.

Untuk pembayaran, sistem memakai lapisan abstraksi *Payment Provider*: implementasi V1 adalah *MockPaymentProvider* (simulasi, tanpa panggilan API eksternal), dan pada tahap

lanjutan akan diganti *TripayPaymentProvider* yang memanggil API *payment gateway* Tripay serta menerima konfirmasi melalui *webhook* bertanda tangan.

Gambar 2.1 menyajikan *Use Case Diagram* yang menggambarkan fitur utama sistem beserta interaksinya dengan para aktor.

[Tempat Gambar 2.1 — Use Case Diagram Sistem MyGerai]

(sisipkan berkas gambar diagram di sini)

Gambar 2.1 Use Case Diagram Sistem MyGerai

Aktor dan *use case* yang tercakup pada diagram tersebut:

- Pembeli: UC-01 Melakukan Pemesanan dan Pembayaran; UC-02 Memantau Status Pesanan.
- Pedagang: UC-03 Mendaftar sebagai Pedagang; UC-04 Mengelola Katalog Item; UC-05 Memproses Pesanan Masuk; UC-06 Mengelola QR Lapak.
- Admin: UC-07 Meninjau Pendaftaran Pedagang; UC-08 Mengatur Konfigurasi Platform; UC-09 Memantau Transaksi dan Saldo Pedagang; UC-10 Mencatat Pencairan Saldo Pedagang.
- *Payment Provider* (sistem eksternal / aktor sekunder): berpartisipasi pada UC-01 untuk pembuatan pembayaran dan mengirim konfirmasi pembayaran (*callback/webhook*) yang memicu perubahan status Pesanan.
- Sistem (proses otomatis, tanpa pemicu manusia): menandai Pesanan *kedaluwarsa*, menghitung Saldo Pedagang, dan mengambil *snapshot* Biaya Layanan serta harga/nama Item.

2.2 Fungsi Produk

Fungsi-fungsi utama dikelompokkan berdasarkan hak akses pengguna dan proses otomatis sistem sebagai berikut.

Fungsi Pembeli

- Memindai QR Lapak dan menelusuri katalog Item satu Lapak tanpa akun.
- Mengelola Keranjang (memilih Item, mengatur kuantitas, menambah catatan) di sisi peramban.
- Melakukan *checkout* dengan mengisi Nama, lalu membayar melalui QRIS (disimulasikan pada V1).
- Memantau Status Pesanan dan melihat Kode Pesanan untuk ditunjukkan saat pengambilan.

Fungsi Pedagang

- Mendaftarkan Lapak secara mandiri (*self-service*) dan menunggu persetujuan Admin.
- Masuk ke *dashboard* menggunakan nomor HP dan kata sandi.
- Mengelola katalog Item: menambah, mengubah, menandai habis atau tersedia.
- Menerima Pesanan masuk yang sudah dibayar secara hampir *real-time* dan memperbarui statusnya (*diproses* → *siap diambil* → *selesai*).
- Mengunduh dan mencetak QR Lapak.

- Melihat akumulasi Saldo Pedagang dan riwayat Pencairan.

Fungsi Admin

- Meninjau pendaftaran Pedagang: menyetujui, atau menolak dengan alasan.
- Mengatur konfigurasi platform: nominal Biaya Layanan dan durasi kedaluwarsa Pesanan, dengan riwayat perubahan.
- Memantau daftar transaksi dan akumulasi Saldo Pedagang per Lapak.
- Mencatat Pencairan manual per Pedagang.

Fungsi Sistem (otomatis)

- Menghitung ulang total Pesanan di server dari harga Item terkini untuk mencegah manipulasi harga.
- Menyimpan *snapshot* Biaya Layanan serta harga dan nama Item pada saat Pesanan dibuat agar histori tidak berubah retroaktif.
- Menandai Pesanan *menunggu_pembayaran* menjadi *kedaluwarsa* saat melewati batas waktu (pengecekan *lazy* ketika Pesanan dibaca).
- Menerima konfirmasi pembayaran dari *Payment Provider* dan mengubah status Pesanan menjadi *dibayar*.
- Menerapkan pembatasan laju (*rate limiting*) pada *checkout*, login, dan pendaftaran.

Penjelasan versi detail setiap fungsi disampaikan pada Bab 3.

2.3 Karakteristik Pengguna

Sistem digunakan oleh empat klasifikasi pengguna yang dibedakan berdasarkan tingkat hak akses dan ruang lingkup fitur.

Klasifikasi Pengguna	Karakteristik dan Tingkat Keahlian	Tingkat Kepentingan
Pembeli	Masyarakat umum yang melintas di sekitar Lapak. Literasi teknologi sangat beragam (dasar hingga menengah), sering memakai ponsel kelas bawah pada jaringan lambat, dan kemungkinan sedang terburu-buru. Tidak memiliki akun. Membutuhkan antarmuka yang sangat sederhana dan alur <i>checkout</i> sesingkat mungkin.	Sangat Penting (Prioritas Utama)
Pedagang	Pemilik usaha kecil informal (tukang bakso, batagor, penjual pakaian, dan sejenisnya). Keahlian teknis sangat beragam, sebagian baru pertama kali memakai aplikasi bisnis. Membutuhkan <i>dashboard</i> pengelolaan Lapak yang lugas dengan tombol aksi yang jelas.	Sangat Penting (Prioritas Utama)
Admin	Pengelola internal Aplikator (pada tahap awal dijalankan langsung oleh pemilik platform). Memahami proses bisnis MyGerai dan terbiasa dengan perkakas administratif berbasis web. Bekerja dari komputer <i>desktop</i> .	Menengah
Aplikator	Pemilik platform MyGerai. Berkepentingan atas kelangsungan bisnis, penerimaan Biaya Layanan,	Menengah

	dan kepatuhan. Berinteraksi dengan sistem terutama melalui peran Admin.	
--	---	--

2.4 Lingkungan Operasional

- Platform: aplikasi web berbasis peramban, *mobile-first*. Tidak ada aplikasi *native* pada V1.
- Peramban yang didukung: versi terbaru Google Chrome (Android), Safari (iOS), Microsoft Edge, dan Mozilla Firefox.
- Perangkat: ponsel pintar untuk Pembeli dan Pedagang; komputer *desktop*/laptop untuk Admin (dan opsional Pedagang).
- Konektivitas: memerlukan koneksi internet. Tidak ada mode *offline*. Halaman Pembeli dioptimalkan untuk jaringan seluler lambat.
- Server: Linux, dijalankan sebagai kontainer Docker melalui Dokploy pada server milik Aplikator (Garuda), diekspos melalui Cloudflare Tunnel.
- Basis data: PostgreSQL (*self-hosted*).
- Teknologi utama: Next.js (*App Router*) + TypeScript + React Server Components, Drizzle ORM, Tailwind CSS, Zod, di atas *runtime* Node.js.

2.5 Batasan Desain dan Implementasi

- *Stack* wajib: Next.js (*App Router*) + TypeScript (*strict*), PostgreSQL *self-hosted* + Drizzle ORM, Tailwind CSS, validasi Zod, *linter*/formatter Biome, paket manajer pnpm, pengujian Vitest (unit) dan Playwright (E2E).
- Arsitektur: satu aplikasi *monolith* (bukan *microservices*); satu basis data terpusat. Tidak ada *background worker* atau *message broker* terpisah pada V1 — tugas terjadwal ringan dilakukan secara *lazy* saat data diakses.
- Basis data hanya diakses dari kode server (*Server Action/Route Handler*); peramban tidak pernah terhubung langsung ke basis data.
- Isolasi antar-Lapak (*multi-tenant*) ditegakkan di level aplikasi — setiap *Server Action* wajib memfilter query berdasarkan identitas Pedagang dari sesi login — bukan melalui *Row Level Security* basis data.
- *Realtime* memakai *polling* berkala dari peramban (interval beberapa detik), bukan *WebSocket* atau *push* instan.
- Pembayaran V1 *disimulasikan* melalui *MockPaymentProvider*; seluruh kode pembayaran tetap ditulis di balik antarmuka *PaymentProvider* agar dapat diganti ke Tripay tanpa mengubah arsitektur. Field *provider* pada tabel *payments* membedakan transaksi *mock* dari *tripay*.
- Metode pembayaran V1 hanya QRIS. Tidak ada tunai, transfer manual, atau dompet digital langsung.
- Autentikasi Pedagang/Admin: nomor HP + kata sandi, tanpa OTP. Kata sandi di-*hash* dengan *scrypt* (*node:crypto*); sesi disimpan di basis data (*DB-backed*), token di *cookie HttpOnly* hanya disimpan sebagai *hash* SHA-256.
- Pembeli tidak memiliki autentikasi apa pun. Identifikasi Pesanan Pembeli memakai *orders.id* (UUID v4) sebagai kredensial akses pada URL.
- Uang selalu dapat dikonfigurasi dan tidak boleh di-*hardcode*; setiap perubahan konfigurasi bisnis tidak boleh mengubah histori (nilai di-*snapshot* per Pesanan).
- Bahasa, lokal, dan zona waktu: seluruh antarmuka Bahasa Indonesia, mata uang Rupiah, zona waktu Asia/Jakarta. Tidak ada dukungan multibahasa.
- Setiap kode yang menyentuh pembayaran, autentikasi, atau *webhook* wajib melewati tinjauan keamanan (*security review*) sebelum dianggap selesai. Perubahan struktur basis data wajib melalui migrasi terversi (Drizzle).

2.6 Asumsi & Dependensi

- Ketersediaan perangkat dan jaringan: seluruh pengguna memiliki ponsel atau komputer dengan peramban modern dan koneksi internet aktif (seluler atau Wi-Fi).
- Literasi digital Pedagang: Pedagang mampu mengisi formulir sederhana, membaca notifikasi, dan menekan tombol aksi secara mandiri.
- Kejujuran ketersediaan Item: Pedagang memperbarui status Item (*available/sold_out*) agar sesuai kondisi nyata di Lapak.
- Ketersediaan *Payment Provider*: pada tahap lanjutan, integrasi bergantung pada ketersediaan dan kebijakan akun Tripay (didaftarkan Aplikator sebagai perorangan dengan KTP). Pada V1 dependensi ini digantikan *MockPaymentProvider*.
- Infrastruktur Aplikator: server Garuda, Dokploy, dan Cloudflare Tunnel tersedia dan dikelola oleh Aplikator; basis data PostgreSQL produksi disiapkan sebelum *go-live*.
- Status badan usaha Aplikator saat ini perorangan; pemilihan *payment gateway* dan batas transaksi mengikuti kondisi ini dan perlu ditinjau ulang bila berkembang menjadi badan hukum.
- Kepatuhan QRIS dan penyelenggaraan sistem pembayaran mengikuti ketentuan Bank Indonesia dan penyedia *payment gateway* berizin; MyGerai tidak menahan dana di luar sistem penyedia tersebut.

3. Fitur Sistem

Bab ini menguraikan secara terperinci fitur utama yang disediakan sistem. Fitur diorganisasikan berdasarkan *Use Case Diagram* pada Bab 2.

Setiap fitur utama didefinisikan sebagai sebuah *use case* dan dijabarkan dengan format *fully dressed*. Setiap penjabaran *use case* diikuti subbab *Functional Requirements* yang memuat daftar spesifikasi teknis terperinci beserta prioritasnya, sebagai acuan bagi tim *developer* dan *tester* mengenai komponen antarmuka maupun algoritma yang harus dibangun untuk merealisasikan fungsi tersebut.

3.1 Fitur 1: Melakukan Pemesanan dan Pembayaran

Atribut	Deskripsi
Kode Use Case	UC-01
Nama Use Case	Melakukan Pemesanan dan Pembayaran
Aktor Utama	Pembeli
Stakeholder & Interest	Pembeli: ingin memesan dengan cepat, tanpa akun, tanpa biaya tersembunyi, dan yakin pesanannya diterima Pedagang. Pedagang: ingin menerima Pesanan yang sudah dibayar tanpa salah catat. Aplikator: memperoleh Biaya Layanan dari setiap Pesanan yang sukses dibayar.
Konteks dan Tujuan	Fitur ini memungkinkan Pembeli memindai QR Lapak, memilih Item dari satu Lapak, mengisi Nama, dan menyelesaikan pembayaran QRIS (disimulasikan pada V1) sehingga tercipta Pesanan yang otomatis diteruskan ke Pedagang setelah pembayaran terkonfirmasi.

Prakondisi (Preconditions)	1. Lapak berstatus <i>approved</i> dan QR Lapak aktif. 2. Pembeli membuka halaman katalog Lapak melalui pemindaian QR (<i>/menu/{slug}</i>). 3. Minimal satu Item pada Lapak berstatus <i>available</i>. 4. Layanan <i>Payment Provider</i> aktif (pada V1, <i>MockPaymentProvider</i> selalu tersedia).
Kondisi Akhir Sukses (Postconditions / Success Guarantee)	1. Sistem merekam Pesanan beserta rinciannya (<i>order_items</i>) dengan <i>merchant_id</i> yang benar. 2. Total dihitung ulang di server; <i>subtotal</i>, <i>platform_fee_snapshot</i>, dan <i>total_for_merchant</i> tersimpan pada Pesanan. 3. Pembayaran terverifikasi; status Pesanan menjadi <i>dibayar</i> dan <i>paid_at</i> terisi. 4. Kode Pesanan diterbitkan dan tampil pada halaman Status Pesanan Pembeli. 5. Pesanan muncul pada <i>dashboard</i> Pedagang; baris <i>payments</i> tercatat dengan <i>provider</i> dan <i>raw_payload</i>.
Skenario Utama (Main Success Scenario)	1. Pembeli memindai QR Lapak dan sistem menampilkan katalog Item Lapak tersebut (hanya Lapak <i>approved</i> dan Item <i>available</i>). 2. Pembeli memilih Item, mengatur kuantitas, dan menambahkan catatan; Item masuk ke Keranjang di sisi peramban. 3. Pembeli memberikan instruksi untuk melanjutkan ke <i>checkout</i>. 4. Sistem menampilkan ringkasan Keranjang dan subtotal, serta formulir isian Nama. 5. Pembeli mengisi Nama dan memberikan instruksi untuk membuat Pesanan. 6. Sistem memvalidasi masukan (Nama wajib), mengambil ulang harga tiap Item dari basis data, dan menghitung ulang subtotal di server. 7. Sistem mengambil nilai Biaya Layanan yang berlaku dari <i>platform_config</i>, menyimpannya sebagai <i>platform_fee_snapshot</i>, dan menghitung <i>total_for_merchant</i>. 8. Sistem membuat Pesanan berstatus <i>menunggu_pembayaran</i>, menetapkan <i>expires_at</i> = <i>created_at</i> + <i>order_expiry_minutes</i>, dan membuat baris <i>order_items</i> dengan <i>snapshot</i> nama serta harga Item. 9. Sistem memanggil <i>PaymentProvider.createPayment()</i> dan menampilkan QRIS pembayaran beserta halaman Status Pesanan.

	10. Pembeli melakukan pembayaran QRIS. Pada V1, Pembeli menekan tombol "Simulasikan Pembayaran Berhasil". 11. Payment Provider mengirim konfirmasi pembayaran sukses ke sistem melalui <i>handleCallback()</i> (pada tahap lanjutan berupa <i>webhook</i> bertanda tangan yang diverifikasi lebih dahulu). 12. Sistem mencocokkan <i>reference_id</i>, mengubah status Pesanan menjadi <i>dibayar</i>, mengisi <i>paid_at</i>, dan menyimpan baris <i>payments</i>. 13. Sistem menampilkan Kode Pesanan pada halaman Status Pesanan Pembeli dan memunculkan Pesanan pada <i>dashboard</i> Pedagang (terlihat setelah siklus <i>polling</i> berikutnya).
Skenario Alternatif (Extensions / Alternative Flows)	Kondisi 1a: Percobaan pemesanan berlebihan dari satu sumber 1. Jumlah pembuatan Pesanan dari satu alamat IP melebihi ambang (20 per 10 menit). 2. Sistem menolak sementara dan menampilkan pesan generik untuk mencoba beberapa saat lagi. Kondisi 2a: Item menjadi habis sebelum Pesanan dibuat 1. Sistem mendeteksi Item berstatus <i>sold_out</i> saat pembuatan Pesanan. 2. Sistem menolak dan menampilkan pesan "Item ini baru saja habis, silakan pilih yang lain". 3. Pembeli menyesuaikan Keranjang dan mengulang dari langkah 3. Kondisi 6a: Nama kosong 1. Sistem menolak pembuatan Pesanan dan menandai kolom Nama sebagai wajib diisi. 2. Pembeli mengisi Nama dan mencoba kembali. Kondisi 6b: Total dari peramban berbeda dengan hitungan server 1. Sistem mengabaikan total yang dikirim klien dan memakai hasil hitung ulang server. 2. Pesanan dibuat dengan nilai server; tidak ada pesan khusus kepada Pembeli. Kondisi 10a: Batas waktu pembayaran terlampaui 1. Saat Pesanan <i>menunggu_pembayaran</i> dibaca setelah <i>expires_at</i>, sistem mengubah statusnya menjadi <i>kedaluwarsa</i>. 2. Halaman Status Pesanan menampilkan status kedaluwarsa dan menawarkan Pembeli untuk memesan ulang. Kondisi 11a: Konfirmasi pembayaran gagal atau ditolak 1. Status Pesanan tetap <i>menunggu_pembayaran</i>.

	2. Sistem menampilkan pesan kegagalan dan memungkinkan Pembeli mencoba lagi hingga <i>expires_at</i>. Kondisi 11b: Konfirmasi pembayaran diterima lebih dari sekali 1. Sistem memproses secara <i>idempotent</i>: bila Pesanan sudah <i>dibayar</i>, konfirmasi berikutnya diabaikan tanpa efek ganda.
--	--

3.1.1 Functional Requirements

ID	Nama Requirement	Deskripsi	Prioritas
FR-01	Halaman Katalog Lapak via QR	Sistem menampilkan Item satu Lapak (nama, harga, foto) berdasarkan <i>slug</i> pada URL QR Lapak; hanya menampilkan Lapak <i>approved</i> dan Item <i>available</i> .	Tinggi
FR-02	Keranjang Sisi Peramban	Sistem memungkinkan Pembeli menambah, mengubah kuantitas, memberi catatan, dan menghapus Item pada Keranjang; Keranjang bertahan di penyimpanan peramban tanpa membuat Pesanan.	Tinggi
FR-03	Indikator Item Habis	Sistem menonaktifkan tombol tambah dan menampilkan label peringatan yang jelas untuk Item berstatus <i>sold_out</i> .	Tinggi
FR-04	Formulir Checkout (Nama)	Sistem menyediakan halaman <i>checkout</i> yang merangkum isi Keranjang dan subtotal serta meminta Nama Pembeli sebagai satu-satunya field wajib.	Tinggi
FR-05	Pembuatan Pesanan di Server	Sistem membuat Pesanan melalui <i>Server Action</i> : memvalidasi input dengan <i>Zod</i> , mengambil ulang harga Item dari basis data, menghitung ulang subtotal, dan menetapkan <i>merchant_id</i> dari <i>slug</i> — tidak mempercayai nilai dari klien.	Tinggi
FR-06	Snapshot Biaya Layanan	Saat Pesanan dibuat, sistem menyalin <i>platform_fee_amount</i> aktif dari <i>platform_config</i> ke <i>orders.platform_fee_snapshot</i> dan menghitung $total_for_merchant = subtotal - platform_fee_snapshot$ (di-clamp minimal 0).	Tinggi
FR-07	Snapshot Item pada Pesanan	Sistem menyimpan <i>product_name_snapshot</i> dan <i>price_snapshot</i> pada setiap baris <i>order_items</i> agar perubahan katalog kemudian tidak mengubah histori Pesanan.	Tinggi

FR-08	Perhitungan Waktu Kedaluwarsa	Sistem menetapkan <i>expires_at</i> = <i>created_at</i> + <i>order_expiry_minutes</i> dari <i>platform_config</i> saat Pesanan dibuat.	Tinggi
FR-09	Pembuatan Pembayaran (Payment Provider)	Sistem memanggil <i>PaymentProvider.createPayment(order)</i> dan menampilkan gambar QRIS beserta <i>referenceId</i> ; V1 memakai <i>MockPaymentProvider</i> tanpa panggilan API eksternal.	Tinggi
FR-10	Tombol Simulasi Pembayaran (V1)	Halaman Status Pesanan menyediakan tombol "Simulasikan Pembayaran Berhasil" yang memanggil <i>handleCallback</i> dengan status sukses.	Tinggi
FR-11	Pemrosesan Konfirmasi Pembayaran	Sistem mencocokkan <i>reference_id</i> , mengubah status Pesanan <i>menunggu_pembayaran</i> → <i>dibayar</i> , mengisi <i>paid_at</i> , dan menyimpan baris <i>payments (provider, raw_payload)</i> ; pemrosesan bersifat <i>idempotent</i> terhadap konfirmasi ganda dan menolak Pesanan yang sudah <i>kedaluwarsa</i> .	Tinggi
FR-12	Verifikasi Tanda Tangan Webhook (tahap lanjutan)	Endpoint <i>/api/webhooks/payment</i> wajib memverifikasi tanda tangan Tripay sebelum memproses payload apa pun.	Menengah
FR-13	Penerbitan Kode Pesanan	Sistem menerbitkan Kode Pesanan pendek (kombinasi huruf dan angka) yang unik per hari per Lapak dan mudah disebutkan secara lisan.	Tinggi
FR-14	Pembatasan Laju Checkout	Sistem membatasi pembuatan Pesanan hingga 20 per 10 menit per alamat IP (IP diambil dari <i>cf-connecting-ip</i> , <i>fallback</i> segmen terakhir <i>x-forwarded-for</i>); pesan saat batas tercapai bersifat generik.	Menengah

3.2 Fitur 2: Memantau Status Pesanan

Atribut	Deskripsi
Kode Use Case	UC-02
Nama Use Case	Memantau Status Pesanan
Aktor Utama	Pembeli

Stakeholder & Interest	Pembeli: ingin mengetahui kapan Pesanan siap diambil dan memiliki penanda (Kode Pesanan) untuk ditunjukkan kepada Pedagang. Pedagang: ingin Pembeli datang mengambil pada saat pesanan benar-benar siap.
Konteks dan Tujuan	Setelah <i>checkout</i> , Pembeli menerima halaman Status Pesanan (URL memuat <i>orderId</i> berupa UUID) yang menampilkan status terkini, rincian Item, total, dan Kode Pesanan, serta diperbarui otomatis melalui <i>polling</i> tanpa pemuatan ulang manual.
Prakondisi (Preconditions)	1. Pesanan sudah dibuat (minimal berstatus <i>menunggu pembayaran</i>). 2. Pembeli memegang URL halaman Status Pesanan (dibuka otomatis setelah <i>checkout</i> pada perangkat yang sama).
Kondisi Akhir Sukses (Postconditions / Success Guarantee)	1. Pembeli melihat status terkini Pesanan. 2. Ketika status <i>siap_diambil</i>, tampilan menonjolkan Kode Pesanan sebagai penanda pengambilan.
Skenario Utama (Main Success Scenario)	1. Sistem menampilkan halaman Status Pesanan berisi status, daftar Item, total, dan Kode Pesanan. 2. Peramban Pembeli melakukan <i>polling</i> berkala (sekitar 4 detik) untuk mengambil status terbaru dari server. 3. Saat Pedagang memperbarui status Pesanan, tampilan Pembeli ikut berubah tanpa pemuatan ulang manual. 4. Ketika status menjadi <i>siap_diambil</i>, sistem menyorot Kode Pesanan. 5. Pembeli menunjukkan atau menyebutkan Kode Pesanan kepada Pedagang saat pengambilan; Pedagang menandai Pesanan <i>selesai</i>.
Skenario Alternatif (Extensions / Alternative Flows)	Kondisi 1a: <i>orderId</i> tidak ditemukan atau tidak valid 1. Sistem menampilkan halaman "Pesanan tidak ditemukan" (404 kustom). Kondisi 2a: Pesanan belum dibayar dan melewati <i>expires_at</i> 1. <i>Polling</i> berikutnya membaca Pesanan dan sistem mengubah statusnya menjadi <i>kedaluwarsa</i>. 2. Halaman menampilkan status kedaluwarsa dan menawarkan tombol untuk memesan kembali. Kondisi 3a: Pembeli menutup lalu membuka kembali halaman di perangkat yang sama 1. Halaman dimuat ulang dari URL yang sama dan menampilkan status terkini. 2. Tidak ada riwayat lintas perangkat karena Pembeli tidak memiliki akun.

3.2.1 Functional Requirements

ID	Nama Requirement	Deskripsi	Prioritas
FR-15	Halaman Status Pesanan	Sistem menyediakan halaman yang menampilkan status, rincian Item, total, dan Kode Pesanan sebuah Pesanan, di-query langsung berdasarkan <i>orders.id</i> melalui <i>Server Component/Route Handler</i> .	Tinggi
FR-16	Pembaruan Status Otomatis	Sistem memperbarui status pada halaman Pembeli melalui <i>polling</i> berkala (sekitar 4 detik) tanpa pemuatan ulang manual.	Tinggi
FR-17	Penonjolan Kode Pesanan	Sistem menampilkan Kode Pesanan secara menonjol, khususnya ketika status Pesanan <i>siap_diambil</i> .	Tinggi
FR-18	Pengecekan Kedaluwarsa Lazy	Setiap kali Pesanan <i>menunggu_pembayaran</i> dibaca dan <i>now() > expires_at</i> , sistem mengubah statusnya menjadi <i>kedaluwarsa</i> saat itu juga.	Tinggi
FR-19	Halaman 404 Kustom	Sistem menampilkan halaman khusus berbahasa Indonesia ketika <i>orderId</i> tidak dikenal atau tidak valid.	Menengah

3.3 Fitur 3: Mendaftar sebagai Pedagang

Atribut	Deskripsi
Kode Use Case	UC-03
Nama Use Case	Mendaftar sebagai Pedagang
Aktor Utama	Pedagang
Stakeholder & Interest	Pedagang: ingin bergabung ke platform tanpa proses berbelit. Aplikator/Admin: ingin kontrol kualitas dasar sebelum QR Lapak aktif untuk publik.
Konteks dan Tujuan	Pedagang mendaftar secara mandiri (<i>self-service</i>) dengan mengisi data Lapak dan pemilik serta membuat kredensial (nomor HP + kata sandi). Lapak dibuat berstatus <i>pending</i> dan menunggu persetujuan Admin sebelum QR Lapak aktif.
Prakondisi (Preconditions)	1. Nomor HP yang dipakai belum terdaftar sebagai Pedagang. 2. Pedagang membuka halaman pendaftaran publik (<i>/daftar</i>).
Kondisi Akhir Sukses (Postconditions / Success Guarantee)	1. Baris <i>merchants</i> dibuat berstatus <i>pending</i> dengan <i>slug</i> unik dan <i>password_hash</i> . 2. QR Lapak belum aktif untuk publik. 3. Pedagang dapat mencoba login tetapi hanya menerima informasi status pendaftaran.

Skenario Utama (Main Success Scenario)	1. Pedagang membuka halaman pendaftaran dan mengisi data Lapak (nama Lapak, kategori, foto/banner) serta data pemilik (nama pemilik, nomor HP, info rekening/e-wallet pencairan). 2. Pedagang menetapkan nomor HP dan kata sandi untuk login. 3. Pedagang memberikan instruksi untuk mengirim pendaftaran. 4. Sistem memvalidasi seluruh masukan dengan Zod dan memastikan nomor HP belum dipakai. 5. Sistem membuat <i>slug</i> unik dari nama Lapak, menambahkan sufiks acak bila terjadi tabrakan. 6. Sistem menyimpan kata sandi sebagai <i>hash scrypt</i> dan membuat baris <i>merchants</i> berstatus <i>pending</i>. 7. Sistem menampilkan konfirmasi bahwa pendaftaran sedang menunggu persetujuan Admin.
Skenario Alternatif (Extensions / Alternative Flows)	Kondisi 4a: Nomor HP sudah terdaftar 1. Sistem menolak pendaftaran dengan pesan generik tanpa mengonfirmasi keberadaan akun secara eksplisit (pola anti-enumerasi). Kondisi 4b: Data tidak lengkap atau format tidak sesuai 1. Sistem mendeteksi kolom wajib yang kosong atau berkas foto yang bukan gambar. 2. Sistem menampilkan pesan galat spesifik pada kolom yang bermasalah. 3. Pedagang memperbaiki masukan dan mengirim kembali. Kondisi 1a: Percobaan pendaftaran berlebihan dari satu sumber 1. Jumlah pendaftaran dari satu alamat IP melebihi ambang (5 per 60 menit). 2. Sistem menolak sementara dengan pesan generik.

3.3.1 Functional Requirements

ID	Nama Requirement	Deskripsi	Prioritas
FR-20	Halaman Pendaftaran Pedagang	Sistem menyediakan halaman publik untuk mengisi data Lapak, data pemilik, kontak, foto, dan info rekening pencairan.	Tinggi
FR-21	Validasi & Keunikan Nomor HP	Sistem memvalidasi masukan dengan Zod dan menolak pendaftaran bila nomor HP sudah terdaftar, dengan pesan generik anti-enumerasi.	Tinggi
FR-22	Generasi Slug Unik	Sistem membuat <i>slug</i> untuk URL QR Lapak dari nama Lapak; bila bentrok, menambahkan sufiks acak dan mencoba ulang.	Tinggi

FR-23	Penyimpanan Kata Sandi Aman	Sistem menyimpan kata sandi sebagai <i>hash script</i> berformat "<saltHex>:<hashHex>"; kata sandi mentah tidak pernah disimpan.	Tinggi
FR-24	Pembuatan Merchant Berstatus Pending	Sistem membuat baris <i>merchants</i> berstatus <i>pending</i> ; QR Lapak tidak aktif hingga <i>approved</i> .	Tinggi
FR-25	Pembatasan Laju Pendaftaran	Sistem membatasi pendaftaran hingga 5 per 60 menit per alamat IP.	Menengah
FR-26	Informasi Status Pasca-pendaftaran	Sistem menampilkan konfirmasi bahwa pendaftaran menunggu persetujuan Admin dan menjelaskan langkah berikutnya.	Menengah

3.4 Fitur 4: Mengelola Katalog Item

Atribut	Deskripsi
Kode Use Case	UC-04
Nama Use Case	Mengelola Katalog Item
Aktor Utama	Pedagang
Stakeholder & Interest	Pedagang: ingin menambah, mengubah, dan menandai habis Item di Lapaknya dengan mudah. Pembeli: ingin melihat informasi Item yang akurat, termasuk harga dan ketersediaan terbaru.
Konteks dan Tujuan	Fitur ini memungkinkan Pedagang mengelola katalog Item Lapaknya sendiri (nama, deskripsi, harga, foto, status ketersediaan) melalui <i>dashboard</i> , dengan seluruh operasi difilter berdasarkan identitas Pedagang dari sesi login.
Prakondisi (Preconditions)	1. Pedagang telah berhasil login dan Lapak berstatus <i>approved</i> . 2. Pedagang berada pada menu pengelolaan Item di <i>dashboard</i> .
Kondisi Akhir Sukses (Postconditions / Success Guarantee)	1. Sistem menyimpan pembaruan data Item pada Lapak yang benar. 2. Item <i>available</i> langsung tampil pada katalog publik Lapak; Item <i>sold_out</i> disembunyikan dari katalog namun datanya tetap ada.
Skenario Utama (Main Success Scenario)	1. Pedagang membuka menu pengelolaan Item pada <i>dashboard</i> . 2. Sistem menampilkan daftar Item milik Lapak tersebut (nama, harga, status) — tidak pernah menampilkan Item milik Lapak lain. 3. Pedagang memberikan instruksi untuk menambah Item baru. 4. Sistem menampilkan formulir isian Item (nama, deskripsi, harga, foto).

	- Pedagang mengisi data dan memberikan instruksi untuk menyimpan. - Sistem memvalidasi masukan dengan Zod (harga berupa bilangan bulat non-negatif, foto berupa gambar). - Sistem menyimpan Item dengan <i>merchant_id</i> dari sesi dan memperbarui daftar pada <i>dashboard</i>. - Pedagang dapat menandai Item <i>sold_out</i> atau <i>available</i>, atau menyunting Item yang ada, kapan pun.
Skenario Alternatif (Extensions / Alternative Flows)	Kondisi 6a: Masukan tidak valid atau tidak lengkap - Sistem mendeteksi kolom wajib kosong atau format tidak sesuai. - Sistem menampilkan pesan galat spesifik pada kolom bermasalah. - Pedagang memperbaiki dan menyimpan kembali. Kondisi 3a: Menyunting Item yang sudah ada - Pedagang memilih Item pada daftar dan memberikan instruksi ubah. - Sistem menampilkan formulir terisi data Item tersebut. - Alur kembali ke langkah 5 pada Skenario Utama. Kondisi 2a: Upaya mengakses Item milik Lapak lain - Sistem menolak karena setiap query difilter <i>WHERE merchant_id = <id dari sesi></i>. - Item milik Lapak lain tidak pernah tampil maupun dapat diubah.

3.4.1 Functional Requirements

ID	Nama Requirement	Deskripsi	Prioritas
FR-27	Login Pedagang & Penjaga Dashboard	Sistem menyediakan login Pedagang (nomor HP + kata sandi) dan menolak akses ke rute <i>dashboard</i> tanpa sesi yang valid.	Tinggi
FR-28	Daftar Item Milik Lapak	Sistem menampilkan daftar seluruh Item milik Lapak yang sedang login, lengkap dengan nama, harga, dan status.	Tinggi
FR-29	Formulir Tambah Item	Sistem menyediakan formulir untuk memasukkan Item baru dengan atribut: nama, deskripsi, harga, dan foto.	Tinggi
FR-30	Ubah Data Item	Sistem menyediakan fungsi untuk menyunting rincian Item yang sudah tersimpan.	Tinggi

FR-31	Ubah Status Ketersediaan Item	Sistem memungkinkan Pedagang menandai Item <i>sold_out</i> atau <i>available</i> tanpa menghapus data Item.	Tinggi
FR-32	Unggah & Optimasi Foto Item	Sistem menerima unggahan foto Item dan menyajikannya melalui <i>next/image (resize</i> dan format modern otomatis).	Menengah
FR-33	Isolasi Kepemilikan per Server Action	Setiap <i>Server Action</i> yang menyentuh <i>products</i> memfilter <i>WHERE merchant_id = <id dari sesi login></i> ; identitas tidak diambil dari parameter klien.	Tinggi

3.5 Fitur 5: Memproses Pesanan Masuk

Atribut	Deskripsi
Kode Use Case	UC-05
Nama Use Case	Memproses Pesanan Masuk
Aktor Utama	Pedagang
Stakeholder & Interest	Pedagang: ingin melihat Pesanan baru yang sudah dibayar, menyiapkan Item, dan menyerahkannya kepada Pembeli dengan benar. Pembeli: ingin Pesannya segera diproses dan siap diambil sesuai rincian.
Konteks dan Tujuan	Fitur ini memungkinkan Pedagang mengelola siklus hidup Pesanan masuk mulai dari menerima Pesanan <i>dibayar</i> , memprosesnya, menandai siap diambil, hingga menandai selesai saat Pembeli mengambil dan menyebut Kode Pesanan.
Prakondisi (Preconditions)	1. Pedagang telah login dan berada pada <i>dashboard</i> Pedagang. 2. Terdapat minimal satu Pesanan berstatus <i>dibayar</i> untuk Lapak tersebut.
Kondisi Akhir Sukses (Postconditions / Success Guarantee)	1. Status Pesanan berpindah maju sesuai aksi Pedagang (<i>dibayar</i> → <i>diproses</i> → <i>siap_diambil</i> → <i>selesai</i>). 2. Pembeli menerima pembaruan status pada halaman Status Pesanan melalui <i>polling</i>. 3. <i>completed_at</i> terisi ketika Pesanan <i>selesai</i>.
Skenario Utama (Main Success Scenario)	1. Pedagang membuka menu Pesanan pada <i>dashboard</i>. 2. Sistem menampilkan daftar Pesanan berstatus <i>dibayar</i> untuk Lapak tersebut, diperbarui melalui <i>polling</i> berkala (sekitar 5 detik). 3. Pedagang memilih sebuah Pesanan untuk melihat rincian Item, kuantitas, catatan, Nama Pembeli, dan Kode Pesanan. 4. Pedagang memberikan instruksi "Terima Pesanan".

	5. Sistem memperbarui status Pesanan menjadi <i>diproses</i> dengan transisi maju saja dan <i>optimistic lock</i>, lalu memberi tahu Pembeli. 6. Pedagang menyiapkan Item sesuai rincian, lalu memberikan instruksi "Tandai Siap Diambil". 7. Sistem memperbarui status menjadi <i>siap_diambil</i>. 8. Pembeli datang dan menyebutkan Kode Pesanan; Pedagang memberikan instruksi "Tandai Selesai". 9. Sistem memperbarui status menjadi <i>selesai</i> dan mengisi <i>completed_at</i>.
Skenario Alternatif (Extensions / Alternative Flows)	Kondisi 5a: Klik ganda atau lompat status 1. Sistem mendeteksi transisi yang tidak berurutan atau perintah berulang. 2. <i>Optimistic lock</i> menolak transisi kedua; hanya satu perubahan yang berlaku. Kondisi 3a: Pesanan sudah <i>kedaluwarsa</i> atau <i>dibatalkan</i> 1. Sistem menonaktifkan tombol aksi status untuk Pesanan tersebut. Kondisi 4a: Koneksi terputus saat mengirim perubahan status 1. Status di server tidak berubah ganda. 2. Tombol aksi kembali aktif setelah kegagalan sehingga Pedagang dapat mencoba lagi.

3.5.1 Functional Requirements

ID	Nama Requirement	Deskripsi	Prioritas
FR-34	Daftar Pesanan Masuk Real-Time	Sistem menampilkan Pesanan berstatus <i>dibayar</i> untuk Lapak yang login pada <i>dashboard</i> , diperbarui melalui <i>polling</i> berkala tanpa refresh manual.	Tinggi
FR-35	Rincian Pesanan	Sistem menyediakan tampilan rincian Pesanan: daftar Item, kuantitas, catatan, total, Nama Pembeli, dan Kode Pesanan.	Tinggi
FR-36	Transisi Status Maju Saja	Sistem hanya mengizinkan transisi status Pesanan ke arah maju (<i>dibayar</i> → <i>diproses</i> → <i>siap_diambil</i> → <i>selesai</i>); tidak ada mundur status.	Tinggi
FR-37	Optimistic Lock Transisi Status	Sistem menerapkan <i>optimistic lock</i> pada pembaruan status untuk mencegah <i>race</i> akibat klik ganda atau lompat status.	Tinggi

FR-38	Pemulihan State Tombol Aksi	Setelah pembaruan status berhasil atau gagal, sistem mengembalikan tombol aksi ke keadaan siap dipakai (mencegah regresi tombol macet di "Memproses...").	Tinggi
FR-39	Penanda Pesanan Baru	Sistem menampilkan penanda visual untuk Pesanan yang baru masuk sejak Pedagang terakhir melihat daftar.	Menengah
FR-40	Isolasi Kepemilikan Pesanan	Setiap <i>Server Action</i> pada <i>orders/order_items</i> memfilter <i>WHERE merchant_id = <id dari sesi login></i> .	Tinggi

3.6 Fitur 6: Mengelola QR Lapak

Atribut	Deskripsi
Kode Use Case	UC-06
Nama Use Case	Mengelola QR Lapak
Aktor Utama	Pedagang
Stakeholder & Interest	Pedagang: ingin memperoleh QR Lapak untuk dicetak dan ditempel di gerobak/lapaknya. Pembeli: ingin memindai QR yang valid dan langsung melihat katalog Lapak.
Konteks dan Tujuan	Fitur ini memungkinkan Pedagang membangkitkan, mengunduh, dan mencetak QR Lapak — QR statis permanen yang mengarah ke halaman katalog Lapak (<i>/menu/{slug}</i>).
Prakondisi (Preconditions)	1. Pedagang telah login. 2. Lapak berstatus <i>approved</i> agar QR aktif untuk publik.
Kondisi Akhir Sukses (Postconditions / Success Guarantee)	1. Pedagang memperoleh berkas gambar QR Lapak (PNG) yang valid untuk dicetak. 2. QR mengarah ke katalog Lapak dan tetap valid meskipun nama Lapak diubah kemudian (karena <i>slug</i> tidak berubah).
Skenario Utama (Main Success Scenario)	1. Pedagang membuka menu QR Lapak pada <i>dashboard</i> . 2. Sistem membangkitkan gambar QR dari URL katalog Lapak menggunakan pustaka <i>qrcode</i> (di sisi server). 3. Sistem menampilkan pratinjau QR beserta URL tujuannya. 4. Pedagang memberikan instruksi untuk mengunduh QR sebagai berkas PNG. 5. Pedagang mencetak dan menempel QR di Lapaknya.
Skenario Alternatif (Extensions / Alternative Flows)	Kondisi 2a: Lapak belum <i>approved</i> 1. Menu QR menampilkan informasi bahwa QR Lapak akan aktif setelah pendaftaran disetujui Admin. 2. Tautan unduh dinonaktifkan atau ditandai belum berlaku.

	Kondisi 1a: Nama Lapak diubah setelah QR dicetak 1. <i>slug</i> dan URL QR tidak berubah; QR yang sudah dicetak tetap valid.
--	--

3.6.1 Functional Requirements

ID	Nama Requirement	Deskripsi	Prioritas
FR-41	Pembangkitan QR Lapak	Sistem membangkitkan gambar QR dari URL katalog Lapak (<i>/menu/{slug}</i>) menggunakan pustaka <i>qrcode</i> di sisi server.	Tinggi
FR-42	Unduh QR sebagai PNG	Sistem menyediakan pengunduhan QR Lapak sebagai berkas gambar PNG yang valid.	Tinggi
FR-43	Cetak QR	Sistem menyediakan tampilan QR yang layak cetak (ukuran memadai, kontras jelas).	Menengah
FR-44	Gating QR pada Status Approved	Sistem hanya mengaktifkan QR Lapak untuk publik setelah Lapak berstatus <i>approved</i> .	Tinggi

3.7 Fitur 7: Meninjau Pendaftaran Pedagang

Atribut	Deskripsi
Kode Use Case	UC-07
Nama Use Case	Meninjau Pendaftaran Pedagang
Aktor Utama	Admin
Stakeholder & Interest	Admin: ingin menyaring pendaftaran Pedagang agar hanya Lapak yang layak yang aktif. Pedagang: ingin kepastian hasil pendaftaran beserta alasannya bila ditolak. Aplikator: ingin kontrol kualitas dasar dan pencegahan penyalahgunaan.
Konteks dan Tujuan	Fitur ini memungkinkan Admin meninjau Pedagang berstatus <i>pending</i> lalu menyetujui atau menolaknya dengan alasan wajib. Persetujuan mengaktifkan QR Lapak; penolakan menyimpan <i>rejection_reason</i> yang ditampilkan kepada Pedagang saat login.
Prakondisi (Preconditions)	1. Admin telah berhasil login (sesi Admin terpisah dari sesi Pedagang). 2. Terdapat minimal satu Pedagang berstatus <i>pending</i> .
Kondisi Akhir Sukses (Postconditions / Success Guarantee)	1. Status Pedagang berubah menjadi <i>approved</i> atau <i>rejected</i> . 2. Pada <i>approved</i> : <i>rejection_reason</i> direset menjadi <i>null</i> dan QR Lapak aktif. 3. Pada <i>rejected</i> : <i>rejection_reason</i> tersimpan dan ditampilkan kepada Pedagang saat mencoba login.

Skenario Utama (Main Success Scenario)	1. Admin membuka daftar Pedagang dan menyaring yang berstatus <i>pending</i>. 2. Admin membuka detail satu Pedagang untuk memeriksa data Lapak dan pemilik. 3. Admin memberikan instruksi "Setujui" atau "Tolak". 4. Untuk "Tolak", sistem meminta Admin mengisi alasan penolakan. 5. Sistem memvalidasi bahwa transisi hanya berasal dari status <i>pending</i> (<i>optimistic lock WHERE status = 'pending'</i>). 6. Sistem memperbarui status Pedagang dan, bila menolak, menyimpan <i>rejection_reason</i>. 7. Sistem menampilkan konfirmasi keberhasilan kepada Admin.
Skenario Alternatif (Extensions / Alternative Flows)	Kondisi 5a: Pedagang sudah tidak berstatus <i>pending</i> 1. Pedagang telah diproses Admin lain atau statusnya sudah berubah. 2. <i>Optimistic lock</i> menyebabkan pembaruan tidak berpengaruh; sistem memberi tahu Admin untuk memuat ulang daftar. Kondisi 4a: Alasan penolakan kosong 1. Sistem menolak aksi dan mewajibkan Admin mengisi alasan sebelum status diubah menjadi <i>rejected</i>.

3.7.1 Functional Requirements

ID	Nama Requirement	Deskripsi	Prioritas
FR-45	Login Admin	Sistem menyediakan login Admin (nomor HP + kata sandi) dengan sesi terpisah (<i>admin_sessions</i> , <i>cookie mygerai_admin_session</i>) yang dapat aktif berdampingan dengan sesi Pedagang.	Tinggi
FR-46	Daftar Pedagang untuk Admin	Sistem menampilkan daftar Pedagang beserta status, dengan penyaring status (termasuk <i>pending</i>).	Tinggi
FR-47	Menyetujui Pedagang	Sistem mengubah status <i>pending</i> → <i>approved</i> , mereset <i>rejection_reason</i> menjadi <i>null</i> , dan mengaktifkan QR Lapak.	Tinggi
FR-48	Menolak Pedagang dengan Alasan Wajib	Sistem mengubah status <i>pending</i> → <i>rejected</i> hanya setelah Admin mengisi <i>rejection_reason</i> .	Tinggi
FR-49	Optimistic Lock Transisi Persetujuan	Sistem menegakkan transisi hanya dari <i>pending</i> melalui klausa <i>WHERE status =</i>	Tinggi

		<i>'pending'</i> untuk mencegah aksi ganda antar-Admin.	
FR-50	Penyampaian Alasan Penolakan ke Pedagang	Sistem menampilkan <i>rejection_reason</i> kepada Pedagang saat mencoba login sehingga tidak perlu kontak terpisah.	Menengah
FR-51	Anti-enumerasi & Pembatasan Laju Login	Login Admin memberi pesan dan waktu respons generik, serta dibatasi 5 percobaan per 5 menit (per-IP dan per-nomor-HP).	Tinggi

3.8 Fitur 8: Mengatur Konfigurasi Platform

Atribut	Deskripsi
Kode Use Case	UC-08
Nama Use Case	Mengatur Konfigurasi Platform
Aktor Utama	Admin
Stakeholder & Interest	Admin/Aplikator: ingin menyesuaikan parameter bisnis (Biaya Layanan, durasi kedaluwarsa) tanpa mengubah kode. Pedagang: mengandalkan nilai Biaya Layanan yang jelas dan histori Saldo yang tidak berubah retroaktif. Pembeli: alur <i>checkout</i> memakai durasi kedaluwarsa yang berlaku.
Konteks dan Tujuan	Fitur ini memberi Admin kewenangan mengubah nilai konfigurasi platform — <i>platform_fee_amount</i> dan <i>order_expiry_minutes</i> — yang disimpan sebagai baris berversi (<i>append-only</i>) dengan <i>effective_from</i> , sehingga perubahan tidak mengubah <i>platform_fee_snapshot</i> Pesanan yang sudah ada.
Prakondisi (Preconditions)	1. Admin telah berhasil login. 2. Sistem terhubung dengan basis data.
Kondisi Akhir Sukses (Postconditions / Success Guarantee)	1. Sistem menyimpan baris <i>platform_config</i> baru hanya untuk key yang nilainya berubah. 2. Nilai aktif adalah baris dengan <i>effective_from</i> $\leq$ <i>now()</i> terbaru untuk tiap key. 3. Pesanan lama tetap memakai <i>platform_fee_snapshot</i> masing-masing.
Skenario Utama (Main Success Scenario)	1. Admin membuka halaman konfigurasi platform. 2. Sistem menampilkan nilai aktif <i>platform_fee_amount</i> dan <i>order_expiry_minutes</i> . 3. Admin mengubah satu atau kedua nilai dan memberikan instruksi untuk menyimpan. 4. Sistem memvalidasi nilai (bilangan bulat non-negatif, durasi minimal 1 menit).

	5. Sistem menyisipkan baris <i>platform_config</i> baru (<i>effective_from</i> = <i>now()</i>) hanya untuk key yang berubah. 6. Sistem menampilkan konfirmasi dan riwayat perubahan.
Skenario Alternatif (Extensions / Alternative Flows)	Kondisi 3a: Tidak ada nilai yang berubah 1. Sistem tidak menyisipkan baris baru dan memberi tahu bahwa tidak ada perubahan. Kondisi 4a: Nilai tidak valid 1. Sistem menolak penyimpanan dan menampilkan pesan galat pada kolom bersangkutan. Kondisi X: Dampak ke Pesanan lama 1. Pesanan yang dibuat sebelum perubahan tetap menyimpan <i>platform_fee_snapshot</i> lama; laporan historis tidak berubah.

3.8.1 Functional Requirements

ID	Nama Requirement	Deskripsi	Prioritas
FR-52	Halaman Konfigurasi Platform	Sistem menyediakan halaman bagi Admin untuk melihat dan mengubah nilai konfigurasi platform.	Tinggi
FR-53	Ubah Nominal Biaya Layanan	Sistem memungkinkan Admin mengubah <i>platform_fee_amount</i> (default 1000).	Tinggi
FR-54	Ubah Durasi Kedaluwarsa Pesanan	Sistem memungkinkan Admin mengubah <i>order_expiry_minutes</i> (default 15).	Tinggi
FR-55	Penyimpanan Konfigurasi Berversi	Sistem menyimpan perubahan sebagai baris baru <i>platform_config</i> dengan <i>effective_from</i> (<i>append-only</i>), tidak melakukan <i>update in place</i> , dan hanya untuk key yang berubah.	Tinggi
FR-56	Pengambilan Konfigurasi Aktif	Sistem menyediakan <i>getActivePlatformConfig()</i> yang memilih baris <i>effective_from</i> ≤ <i>now()</i> terbaru per key; fungsi ini tanpa gerbang Admin karena juga dipakai alur <i>checkout</i> Pembeli dan tidak mengembalikan data sensitif.	Tinggi
FR-57	Validasi Nilai Konfigurasi	Sistem menolak nilai negatif, nonangka, atau durasi di bawah 1 menit.	Menengah
FR-58	Imutabilitas Snapshot Pesanan Lama	Perubahan konfigurasi tidak mengubah <i>platform_fee_snapshot</i> Pesanan yang sudah ada.	Tinggi

3.9 Fitur 9: Memantau Transaksi dan Saldo Pedagang

Atribut	Deskripsi
Kode Use Case	UC-09
Nama Use Case	Memantau Transaksi dan Saldo Pedagang
Aktor Utama	Admin
Stakeholder & Interest	Admin/Aplikator: ingin memantau pergerakan transaksi dan mengetahui kewajiban Pencairan ke tiap Pedagang. Pedagang: ingin Saldo-nya dihitung akurat dari Pesanan yang sukses dikurangi Pencairan yang sudah dilakukan.
Konteks dan Tujuan	Fitur ini memungkinkan Admin melihat daftar transaksi dan akumulasi Saldo Pedagang per Lapak. Saldo Pedagang adalah nilai turunan yang dihitung di server, bukan kolom tersendiri.
Prakondisi (Preconditions)	1. Admin telah berhasil login. 2. Terdapat data Pesanan dan/atau Pencairan.
Kondisi Akhir Sukses (Postconditions / Success Guarantee)	1. Sistem menampilkan metrik transaksi, daftar Pesanan, dan Saldo Pedagang per Lapak secara akurat. 2. Transaksi <i>provider = mock</i> dapat dibedakan dari <i>provider = tripay</i> .
Skenario Utama (Main Success Scenario)	1. Admin membuka halaman transaksi dan Pencairan. 2. Sistem menampilkan daftar Pesanan beserta status, nilai, dan <i>provider</i> . 3. Sistem menghitung Saldo Pedagang per Lapak: $SUM(orders.total_for_merchant$ untuk status <i>dibayar</i> , <i>diproses</i> , <i>siap_diambil</i> , <i>selesai</i>) dikurangi $SUM(payouts.amount$ untuk status <i>selesai</i>). 4. Admin menerapkan penyaring rentang waktu pada daftar. 5. Sistem memperbarui tampilan sesuai rentang yang dipilih.
Skenario Alternatif (Extensions / Alternative Flows)	Kondisi 2a: Tidak ada transaksi pada rentang/Lapak 1. Sistem menampilkan pesan bahwa data transaksi masih kosong. Kondisi 4a: Pencarian Lapak spesifik 1. Admin memasukkan nama Lapak pada kolom pencarian. 2. Sistem menyaring daftar dan menampilkan hanya Lapak yang relevan.

3.9.1 Functional Requirements

ID	Nama Requirement	Deskripsi	Prioritas
FR-59	Daftar Transaksi	Sistem menampilkan daftar Pesanan/pembayaran beserta status, nilai, waktu, dan <i>provider</i> .	Tinggi

FR-60	Pemisahan Transaksi Simulasi dan Nyata	Sistem membedakan dan/atau menyaring transaksi <i>provider = mock</i> dari <i>provider = tripay</i> pada seluruh laporan keuangan.	Tinggi
FR-61	Perhitungan Saldo Pedagang	Sistem menghitung Saldo Pedagang per Lapak di server sebagai nilai turunan dari <i>orders.total_for_merchant</i> dikurangi <i>payouts.amount</i> berstatus <i>selesai</i> ; tidak ada kolom saldo yang bisa <i>out-of-sync</i> .	Tinggi
FR-62	Penyaring Rentang Waktu	Sistem menyediakan penyaring rentang waktu pada halaman laporan.	Menengah
FR-63	Pencarian Lapak	Sistem menyediakan kolom pencarian untuk menemukan Lapak tertentu berdasarkan nama.	Menengah

3.10 Fitur 10: Mencatat Pencairan Saldo Pedagang

Atribut	Deskripsi
Kode Use Case	UC-10
Nama Use Case	Mencatat Pencairan Saldo Pedagang
Aktor Utama	Admin
Stakeholder & Interest	Admin/Aplikator: ingin mencatat Pencairan yang sudah ditransfer ke Pedagang secara akurat, tanpa risiko membayar lebih dari Saldo. Pedagang: ingin Pencairan tercatat benar sehingga Saldo berikutnya akurat.
Konteks dan Tujuan	Pada V1, Pencairan dilakukan Admin secara manual di luar sistem, lalu dicatat di MyGera. Sistem menjaga agar nilai Pencairan tidak melebihi Saldo Pedagang dan aman dari <i>race submit</i> ganda.
Prakondisi (Preconditions)	1. Admin telah berhasil login. 2. Pedagang memiliki Saldo positif. 3. Transfer dana telah dilakukan di luar sistem.
Kondisi Akhir Sukses (Postconditions / Success Guarantee)	1. Baris <i>payouts</i> tercatat (<i>merchant_id</i> , <i>amount</i> , <i>note</i> , status <i>selesai</i> , <i>settled_at</i>). 2. Saldo Pedagang (nilai turunan) otomatis berkurang sebesar <i>amount</i> .
Skenario Utama (Main Success Scenario)	1. Admin membuka halaman Pencairan untuk sebuah Pedagang dan melihat Saldo saat ini. 2. Admin memasukkan nominal Pencairan dan catatan. 3. Admin memberikan instruksi untuk mencatat Pencairan.

	4. Sistem memvalidasi nominal (bilangan bulat positif, tidak melebihi Saldo Pedagang saat itu) di server. 5. Sistem membungkus operasi dalam transaksi basis data dengan <i>advisory lock</i> per-Pedagang. 6. Sistem menyimpan baris <i>payouts</i> berstatus <i>selesai</i> dan mengisi <i>settled_at</i>. 7. Sistem menampilkan konfirmasi dan riwayat Pencairan terbaru.
Skenario Alternatif (Extensions / Alternative Flows)	Kondisi 4a: Nominal melebihi Saldo Pedagang 1. Sistem menolak di server meskipun atribut HTML <i>max</i> pada formulir dihapus melalui manipulasi klien. Kondisi 5a: Dua submit Pencairan bersamaan 1. <i>Advisory lock</i> per-Pedagang menyebabkan hanya satu submit yang berhasil; submit kedua ditolak setelah mengevaluasi ulang Saldo. Kondisi 4b: Nominal nol atau negatif 1. Sistem menolak dengan pesan galat.

3.10.1 Functional Requirements

ID	Nama Requirement	Deskripsi	Prioritas
FR-64	Formulir Catat Pencairan Manual	Sistem menyediakan formulir bagi Admin untuk mencatat Pencairan (<i>amount</i> , <i>note</i>) per Pedagang.	Tinggi
FR-65	Guard Overpayment di Server	Sistem menegakkan $amount \leq Saldo Pedagang$ di sisi server, bukan hanya melalui atribut HTML <i>max</i> .	Tinggi
FR-66	Transaksi & Advisory Lock per Pedagang	Sistem membungkus pencatatan Pencairan dalam transaksi basis data dengan <i>pg_advisory_xact_lock</i> per-Pedagang untuk menutup <i>race TOCTOU</i> submit ganda.	Tinggi
FR-67	Riwayat Pencairan	Sistem menampilkan riwayat Pencairan per Pedagang beserta nominal, catatan, dan waktu.	Menengah
FR-68	Pembaruan Saldo Otomatis	Karena Saldo Pedagang adalah nilai turunan, pencatatan Pencairan langsung tercermin pada perhitungan Saldo berikutnya tanpa pembaruan kolom terpisah.	Tinggi

4. Non-Functional Requirements

4.1 Performance Requirements

ID	Parameter Kinerja	Deskripsi Kebutuhan	Rasionalisasi
NFR-01	Waktu Muat Halaman Pembeli	Halaman katalog Lapak (<i>/menu/[slug]</i>) termuat penuh dalam ≤ 3 detik pada simulasi jaringan seluler lambat (setara "Slow 4G" dengan <i>CPU throttle</i> 4×). Skor Lighthouse <i>Performance</i> untuk halaman katalog ≥ 90 .	Pembeli mengakses lewat pindai QR di jalan atau pasar dari ponsel kelas bawah; pemuatan lambat membuat Pembeli membatalkan.
NFR-02	Responsivitas Checkout	<i>Server Action</i> pembuatan Pesanan (validasi, hitung ulang, <i>snapshot</i> , pembuatan pembayaran) selesai < 2 detik pada kondisi normal.	Jeda panjang memicu keraguan atau klik ganda tepat saat Pembeli hendak membayar.
NFR-03	Latensi Notifikasi Pedagang	Pesanan berstatus <i>dibayar</i> tampil di <i>dashboard</i> Pedagang ≤ 6 detik (satu siklus <i>polling</i>) setelah pembayaran terkonfirmasi.	Pedagang perlu segera menyiapkan pesanan; keterlambatan wajar dapat diterima pada skala kaki lima, tetapi tidak boleh lebih dari beberapa detik.
NFR-04	Beban Aset Halaman Pembeli	Halaman Pembeli meminimalkan JavaScript: React Server Components secara default, " <i>use client</i> " hanya untuk bagian interaktif; foto Item/Lapak disajikan melalui <i>next/image</i> .	Menjaga waktu muat dan pemakaian kuota data tetap rendah pada perangkat dan jaringan terbatas.
NFR-05	Kapasitas	Sistem menangani setidaknya 200 pengguna aktif bersamaan dan 100 pembuatan Pesanan per menit tanpa kegagalan atau <i>timeout</i> basis data pada <i>deployment</i> satu instance.	Skala target adalah banyak Lapak kecil dengan trafik moderat, bukan lonjakan kelas <i>e-commerce</i> nasional; angka dipilih realistis untuk satu server.

4.2 Safety Requirements

ID	Parameter Keselamatan	Deskripsi Kebutuhan	Rasionalisasi
----	-----------------------	---------------------	---------------

NFR-06	Konsistensi Transaksi Uang	Operasi yang menyentuh uang (pembuatan Pesanan beserta <i>snapshot</i> , pencatatan pembayaran, pencatatan Pencairan) dijalankan dalam transaksi basis data; kegagalan di tengah proses membatalkan seluruh perubahan (<i>rollback</i>).	Mencegah Pesanan tanpa <i>platform_fee_snapshot</i> , pembayaran ganda, atau Pencairan yang tercatat sebagian.
NFR-07	Imutabilitas Histori Keuangan	Nilai Biaya Layanan serta harga dan nama Item yang berlaku saat Pesanan dibuat disimpan sebagai <i>snapshot</i> pada Pesanan/ <i>order_items</i> dan tidak berubah ketika konfigurasi atau katalog diubah kemudian; <i>platform_config</i> bersifat <i>append-only</i> .	Laporan keuangan dan Saldo Pedagang historis harus tetap dapat direproduksi.
NFR-08	Pemisahan Data Simulasi dan Nyata	Setiap baris <i>payments</i> menyimpan <i>provider (mock/tripay)</i> ; seluruh laporan keuangan wajib memisahkan atau menyaring transaksi <i>mock</i> setelah integrasi nyata aktif.	Mencegah uang simulasi tercampur ke perhitungan pendapatan nyata setelah <i>go-live</i> .
NFR-09	Idempotensi Konfirmasi Pembayaran	Pemrosesan <i>handleCallback/webhook</i> bersifat <i>idempotent</i> : konfirmasi berulang untuk Pesanan yang sudah <i>dibayar</i> tidak menimbulkan efek ganda.	<i>Payment gateway</i> dapat mengirim <i>callback</i> lebih dari sekali; Pesanan tidak boleh terbayar atau tercatat ganda.
NFR-10	Kedaluwarsa Pesanan yang Aman	Pesanan <i>menunggu_pembayaran</i> yang melewati <i>expires_at</i> diubah menjadi <i>kedaluwarsa</i> sebelum aksi lain diproses; pembayaran atas Pesanan kedaluwarsa tidak mengubahnya menjadi <i>dibayar</i> .	Mencegah pesanan basi masuk ke antrean kerja Pedagang.

4.3 Security Requirements

ID	Parameter Keamanan	Deskripsi Kebutuhan	Rasionalisasi
NFR-11	Enkripsi Transport (HTTPS/TLS)	Seluruh lalu lintas data antara peramban dan server memakai	Mencegah penyadapan data Pesanan dan

		TLS 1.2 atau lebih baru (determinasi di Cloudflare).	kredensial di tengah jalan (<i>man-in-the-middle</i>).
NFR-12	Keamanan Kredensial	Kata sandi Pedagang/Admin tidak pernah disimpan sebagai teks terang; di-hash dengan <i>script (node:crypto)</i> berformat " <i><saltHex>:<hashHex></i> " dan diverifikasi dengan <i>crypto.timingSafeEqual</i> . Token sesi hanya disimpan sebagai <i>hash</i> SHA-256, bukan token mentah.	Kebocoran basis data tidak langsung membobol akun atau sesi aktif.
NFR-13	Isolasi Multi-tenant di Level Aplikasi	Setiap <i>Server Action</i> yang membaca/menulis <i>products</i> , <i>orders</i> , <i>order_items</i> , atau <i>payments</i> milik Pedagang wajib memfilter <i>WHERE merchant_id = <id dari sesi login></i> ; identitas diambil dari sesi, bukan parameter klien.	Satu Pedagang tidak boleh melihat atau mengubah data Pedagang lain; bug di satu halaman tidak boleh membocorkan data lintas-Lapak.
NFR-14	Otorisasi Berbasis Peran di Sisi Server	Aksi Admin (konfigurasi, Pencairan, approve/reject Pedagang) hanya dijalankan bila sesi Admin valid (<i>getAdminSession()</i>); sesi Pedagang dan Admin dipisah (tabel dan <i>cookie</i> berbeda).	Mencegah eskalasi hak akses; Pembeli atau Pedagang tidak boleh memanggil fungsi Admin.
NFR-15	Validasi Masukan di Server	Semua masukan dari klien divalidasi dengan Zod di <i>Server Action</i> ; total Pesanan selalu dihitung ulang di server dari <i>products</i> terkini, tidak pernah dipercaya dari payload klien.	Mencegah manipulasi harga dan data cacat masuk ke basis data.
NFR-16	Verifikasi Webhook Pembayaran	Endpoint <i>webhook</i> pembayaran (tahap lanjutan) wajib memverifikasi tanda tangan penyedia sebelum memproses payload; tidak ada kepercayaan pada payload yang belum terverifikasi.	<i>Webhook</i> adalah permukaan serang; payload palsu dapat menandai Pesanan lunas secara curang.
NFR-17	Pembatasan Laju	Diterapkan pada <i>loginMerchant/loginAdmin</i> (5	Menahan <i>brute-force</i> dan <i>spam</i> sederhana

		percobaan / 5 menit, per-IP dan per-nomor-HP), <i>registerMerchant</i> (5 / 60 menit per-IP), dan <i>createOrder</i> (20 / 10 menit per-IP). Alamat IP diambil dari <i>cf-connecting-ip</i> (tidak dapat dipalsukan klien), <i>fallback</i> segmen terakhir <i>x-forwarded-for</i> . Pesan saat batas tercapai bersifat generik.	tanpa infrastruktur tambahan; pemilihan header IP yang keliru dapat dilewati penyerang.
NFR-18	Anti-enumerasi Akun	Login dan pendaftaran memberi pesan dan waktu respons yang seragam untuk akun yang ada maupun tidak ada; login dengan kata sandi benar tetapi status belum <i>approved</i> tidak membuat sesi.	Mencegah penyerang memetakan nomor HP yang terdaftar.
NFR-19	Kerahasiaan Data Sensitif pada Respons	<i>password_hash</i> , <i>token_hash</i> , dan kredensial lain tidak pernah ikut <i>ter-return</i> dari <i>Server Action</i> ke klien maupun <i>bundle</i> sisi klien; respons memakai tipe hasil eksplisit, bukan baris basis data mentah.	Mencegah kebocoran melalui payload respons atau berkas <i>source</i> peramban.
NFR-20	Akses Halaman Status Pesanan	Halaman Status Pesanan Pembeli mengandalkan <i>orders.id</i> (UUID v4, praktis tidak dapat ditebak) sebagai kredensial akses, di-query langsung <i>by primary key</i> dari kode server; Pembeli tidak pernah terhubung langsung ke basis data.	Memberi Pembeli akses tanpa akun tanpa membuka enumerasi Pesanan.

4.4 Software Quality Attributes

ID	Parameter Kualitas	Deskripsi Kebutuhan	Rasionalisasi
NFR-21	Learnability	Pembeli dapat menuntaskan alur pindai → pilih → <i>checkout</i> → bayar dalam ≤ 5 ketukan dari halaman katalog tanpa panduan; Pedagang dapat menambah Item atau memproses Pesanan tanpa buku manual.	Pengguna beragam usia dan literasi digital; kurva belajar harus seminimal mungkin.

NFR-22	Usability & Aksesibilitas Praktis	Ukuran tombol dan teks memadai untuk layar ponsel kecil; kontras warna cukup (tidak ada teks abu tipis di latar putih); alur <i>checkout</i> memiliki langkah sesedikit mungkin.	Pembeli sering terburu-buru dan memakai perangkat kecil di luar ruangan.
NFR-23	Availability	Target ketersediaan layanan bulanan $\geq 99\%$; pemeliharaan terjadwal dilakukan di luar jam sibuk operasional pasar.	Transaksi terjadi setiap hari; <i>downtime</i> pada jam aktif menghentikan pemasukan Pedagang.
NFR-24	Maintainability	Arsitektur modular; parameter bisnis (Biaya Layanan, durasi kedaluwarsa) diisolasi pada <i>platform_config</i> , bukan <i>hardcode</i> . Logika perhitungan uang diekstrak menjadi fungsi <i>pure</i> yang teruji unit.	Perubahan kebijakan dapat diterapkan tanpa kompilasi ulang; perhitungan uang harus mudah diverifikasi.
NFR-25	Portability / Deployability	Aplikasi dikemas sebagai kontainer Docker (<i>output: "standalone"</i>) dan dapat dijalankan pada server mana pun yang mendukung Docker dan PostgreSQL, tanpa ketergantungan pada layanan cloud tertentu.	Menghindari <i>lock-in</i> dan memudahkan pemindahan atau penggandaan lingkungan.
NFR-26	Interoperability	Integrasi dengan <i>payment gateway</i> memakai HTTP + JSON (REST) melalui antarmuka <i>PaymentProvider</i> sehingga penggantian penyedia tidak mengubah arsitektur inti.	Memudahkan migrasi dari <i>MockPaymentProvider</i> ke Tripay atau penyedia lain.
NFR-27	User Error Protection	Sistem menampilkan dialog konfirmasi sebelum aksi yang sulit dibatalkan, misalnya menolak pendaftaran Pedagang, menandai Item habis, atau mencatat Pencairan.	Mengurangi kesalahan akibat klik tidak sengaja pada pengguna yang belum terbiasa.
NFR-28	Recoverability Keranjang	Isi Keranjang bertahan pada penyimpanan peramban sehingga Pembeli yang tidak sengaja menutup halaman sebelum	Mengurangi friksi dan pesanan yang batal.

		<i>checkout</i> dapat melanjutkan tanpa memilih ulang Item.	
NFR-29	Testability	Terdapat lingkungan pengujian terpisah dari produksi (basis data <i>mygerai_test</i>); alur kritikal (<i>checkout</i> Pembeli, terima dan selesaikan Pesanan Pedagang, pembatasan laju) tercakup uji E2E Playwright, dan perhitungan uang tercakup uji unit Vitest.	Perubahan dapat diverifikasi menyeluruh tanpa menyentuh data nyata.
NFR-30	Observability (tahap lanjutan)	Kesiapan integrasi pemantauan galat (misalnya Sentry <i>free tier</i>) ditambahkan setelah MVP berjalan; galat bug bersifat "gagal keras", galat input pengguna bersifat "gagal ramah" dengan pesan Bahasa Indonesia.	Memudahkan deteksi masalah produksi tanpa membebani rilis awal.

4.5 Business Rules

- Biaya Layanan: default Rp1.000 per Pesanan berstatus *dibayar*, dapat dikonfigurasi Admin, dan dipotong dari bagian Pedagang — bukan ditambahkan ke tagihan Pembeli. Nilai yang berlaku di-*snapshot* per Pesanan.
- Model settlement Agregator: seluruh pembayaran masuk ke satu akun milik Aplikator; Pedagang tidak memiliki akun *payment gateway* sendiri; dana diteruskan melalui Pencairan.
- Saldo Pedagang adalah nilai turunan: $SUM(\text{orders.total_for_merchant})$ untuk status *dibayar*, *diproses*, *siap_diambil*, *selesai* dikurangi $SUM(\text{payouts.amount})$ untuk status *selesai*. Tidak ada kolom saldo tersendiri.
- Kedaluwarsa Pesanan: default 15 menit sejak dibuat bila belum *dibayar*; dapat dikonfigurasi Admin.
- Persetujuan Pedagang: QR Lapak hanya aktif untuk publik setelah Lapak berstatus *approved*. Penolakan pendaftaran wajib disertai *rejection_reason* yang ditampilkan kepada Pedagang.
- Transisi Status Pesanan bersifat maju saja: *menunggu pembayaran* → *dibayar* → *diproses* → *siap_diambil* → *selesai*; *dibatalkan* dan *kedaluwarsa* adalah kondisi akhir. Tidak ada mundur status.
- Pembeli tanpa akun: satu-satunya identitas adalah field Nama (bebas diisi, tidak diverifikasi); Kode Pesanan adalah penanda utama saat pengambilan.
- Asumsi 1 Pedagang = 1 Lapak pada V1.
- Metode pembayaran V1 hanya QRIS, dan pada V1 QRIS tersebut disimulasikan.
- Uang tidak boleh di-*hardcode*; perubahan nilai default-nya wajib disetujui Aplikator.
- Kode yang menyentuh pembayaran, autentikasi, atau *webhook* wajib melewati *security review* sebelum dirilis.

5. Requirements Lainnya

- Kepatuhan Pembayaran: Pemrosesan QRIS dan penyelesaian dana dilakukan melalui penyedia *payment gateway* berizin (Tripay pada tahap lanjutan); Aplikator tidak menahan

dana di luar sistem penyedia berizin. Model Agregator perlu ditinjau ulang bila skala transaksi membesar atau status badan usaha Aplikator berubah.

- **Perlindungan Data Pribadi:** Data pribadi yang dikumpulkan diminimalkan — Pembeli hanya Nama; Pedagang menyertakan nomor HP dan info rekening pencairan. Data sensitif tidak ditampilkan kepada pihak yang tidak berkepentingan, dan penanganannya mengikuti Undang-Undang Nomor 27 Tahun 2022 tentang Pelindungan Data Pribadi.
- **Retensi Data:** Data Pesanan, pembayaran, dan Pencairan tidak dihapus permanen (*hard delete*) dari basis data produksi; Item yang habis ditandai *sold out*, bukan dihapus, agar histori Pesanan tetap valid melalui *snapshot* pada *order items*.
- **Kepatuhan Perdagangan Melalui Sistem Elektronik:** Sistem mendukung penyajian identitas Lapak (nama Lapak, kategori, kontak) secara jelas kepada Pembeli, sejalan dengan Peraturan Pemerintah Nomor 80 Tahun 2019 tentang Perdagangan Melalui Sistem Elektronik.
- **Internasionalisasi:** V1 hanya mendukung Bahasa Indonesia dan mata uang Rupiah; tidak ada infrastruktur i18n yang dibangun.
- **Lisensi Perangkat Lunak:** Seluruh pustaka pihak ketiga yang dipakai berlisensi permisif (MIT atau setara) yang mengizinkan penggunaan komersial tanpa kewajiban membuka kode sumber MyGerai.
- **Kebutuhan Basis Data:** Setiap perubahan skema basis data produksi melalui migrasi terversi (Drizzle) yang dapat dilacak riwayatnya dan diverifikasi pada basis data nyata sebelum diterapkan ke produksi.
- **Ketertelusuran:** Setiap kebutuhan fungsional (FR-XX) dapat ditelusuri ke minimal satu *use case* pada Bab 3, dan setiap kebutuhan non-fungsional (NFR-XX) memiliki rasionalisasi. Perubahan ground truth proyek dicatat pada CHANGELOG.md.

Lampiran A: Glosarium

Istilah pada tabel berikut bersifat baku untuk seluruh dokumen ini, kode, dan antarmuka MyGerai. Definisi domain mengikuti GLOSSARY.md proyek.

Istilah	Definisi / Penjelasan
Aplikator	Pihak pengelola platform MyGerai (pemilik bisnis). Memungut Biaya Layanan dari tiap Pesanan sukses dan bertanggung jawab atas Pencairan ke Pedagang.
Admin	Akun pengelola internal Aplikator. Menyetujui Pedagang baru, mengatur konfigurasi, dan memantau transaksi serta Pencairan.
Pedagang	Pemilik usaha kecil (tukang bakso, batagor, penjual pakaian, dan sejenisnya) yang berjualan lewat MyGerai. Setara <i>merchant/seller</i> .
Lapak	Unit usaha milik satu Pedagang di dalam sistem; memiliki nama, kategori, dan satu QR unik. Asumsi V1: 1 Pedagang = 1 Lapak.
Pembeli	Orang yang memindai QR Lapak dan memesan. Tidak memiliki akun; hanya mengisi Nama saat <i>checkout</i> .
Item	Barang atau menu yang dijual sebuah Lapak (makanan maupun non-makanan). Setara <i>product</i> .
Keranjang	Kumpulan Item yang dipilih Pembeli sebelum <i>checkout</i> , tersimpan sementara di penyimpanan peramban.

Pesanan	Transaksi resmi yang tercipta saat Pembeli <i>checkout</i> . Memiliki Kode Pesanan dan Status Pesanan. Setara <i>order</i> .
Kode Pesanan	Kode pendek unik (misalnya <i>B231</i>) yang ditunjukkan Pembeli kepada Pedagang saat mengambil Pesanan; unik per hari per Lapak.
Status Pesanan	Salah satu dari: <i>menunggu_pembayaran</i> , <i>dibayar</i> , <i>diproses</i> , <i>siap_diambil</i> , <i>selesai</i> , <i>dibatalkan</i> , <i>kedaluwarsa</i> .
QR Lapak	QR statis permanen milik satu Lapak yang mengarah ke halaman katalog Lapak (<i>/menu/{slug}</i>). Dicitak dan ditempel Pedagang.
QRIS Dinamis	QR pembayaran unik per Pesanan dengan nominal sesuai total belanja. Pada V1 disimulasikan.
Payment Provider	Lapisan abstraksi kode untuk pembayaran. Implementasi V1: <i>MockPaymentProvider</i> ; implementasi lanjutan: <i>TripayPaymentProvider</i> .
Biaya Layanan	Potongan yang dipungut Aplikator dari tiap Pesanan sukses. Default Rp1.000, dapat dikonfigurasi. Setara <i>platform fee / flat fee</i> .
Saldo Pedagang	Nilai turunan (bukan kolom) berupa akumulasi hak Pedagang setelah dikurangi Biaya Layanan dan Pencairan yang sudah selesai.
Pencairan (Payout)	Proses Aplikator mentransfer Saldo Pedagang ke rekening/e-wallet Pedagang. V1: dicatat manual oleh Admin.
Model Agregator	Model settlement: semua pembayaran masuk ke satu akun Aplikator, lalu didistribusikan ke Pedagang lewat Pencairan.
Server Action	Fungsi Next.js yang berjalan di server dan menjadi satu-satunya jalur tulis/baca ke basis data dari alur pengguna.
Route Handler	Endpoint HTTP Next.js; dipakai antara lain untuk <i>polling</i> status dan (tahap lanjutan) <i>webhook</i> pembayaran.
Polling	Teknik pembaruan data dengan meminta ulang ke server secara berkala dari peramban, sebagai pengganti koneksi <i>push</i> instan.
React Server Component (RSC)	Komponen React yang dirender di server tanpa mengirim JavaScript komponen tersebut ke peramban; dipakai agar halaman Pembeli ringan.
Snapshot	Penyalinan nilai (harga Item, nama Item, Biaya Layanan) ke dalam Pesanan pada saat dibuat agar histori tidak berubah bila sumbernya berubah.
Lazy Check	Pengecekan kondisi (misalnya kedaluwarsa Pesanan) yang dilakukan saat data diakses, bukan oleh proses terjadwal terpisah.

Idempoten	Sifat operasi yang menghasilkan keadaan akhir sama meskipun dijalankan lebih dari sekali dengan masukan yang sama.
Optimistic Lock	Mekanisme mencegah <i>race</i> dengan menyertakan syarat keadaan lama pada perintah tulis, sehingga hanya satu perubahan bersamaan yang berhasil.
Rate Limiting	Pembatasan jumlah permintaan dari satu sumber dalam rentang waktu tertentu untuk menahan <i>spam</i> dan <i>brute-force</i> .
Webhook	Permintaan HTTP POST dari <i>payment gateway</i> ke server MyGera untuk memberi tahu hasil pembayaran (tahap lanjutan; wajib diverifikasi tanda tangannya).
scrypt	Fungsi <i>hash</i> kata sandi yang lambat secara sengaja, dipakai MyGera dari modul <i>node:crypto</i> untuk menyimpan <i>password_hash</i> .
Slug	Potongan teks ramah-URL yang mengidentifikasi Lapak pada alamat katalog dan QR Lapak.
ESB Order	Produk pemesanan berbasis QR untuk restoran (ekosistem ESB) yang menjadi inspirasi alur inti MyGera.
QRIS	<i>QR Code Indonesian Standard</i> : standar nasional kode QR pembayaran di Indonesia yang diatur Bank Indonesia.